Vanloka Node API — Architecture

This document describes the **Express + TypeScript** API in `application-backend/` (package name `loka-node`): how it is structured, how multi-tenancy and auth work, and how it integrates with MongoDB and AWS.

For a **route-by-route map** (including parity notes vs the Laravel app), see API_ROUTES.pdf.

For **recommended follow-up work** (consistency, security, testing, ops), see IMPROVEMENTS.pdf.

---

1. Purpose

The Node API backs **Vanloka** fleet-management flows: central administration (organisations, masters, app users, devices, plans, roles), **tenant-scoped** operations (vehicles, drivers, bookings, employees, live tracking), and **end-user app** APIs under `/api/user-app`. It is designed to run as a single HTTP service that can also serve a built SPA from `frontend/dist`.

---

2. Technology stack

Layer	Choice
Runtime	Node.js (ES modules, `type: "module"`)
Language	TypeScript → `dist/` via `tsc`
HTTP	Express 4
Process model	One long-lived `app.listen` process; optional separate **SQS worker**
Data	MongoDB via **Mongoose 8**, separate DBs per tenant
Validation / types	Zod in places; many handlers validate inline
API docs	OpenAPI spec in code → `swagger-ui-express` at `/api/docs`
Security middleware	`helmet`, `cors`, `cookie-parser`, `morgan`

Key npm scripts: `dev` (`tsx watch src/app.ts`), `start` (`node dist/app.js`), `worker` (notification worker), `build` (frontend + `tsc`).

---

3. High-level system view

```
flowchart TB
  subgraph clients [Clients]
    Web[Web SPA]
    Admin[Admin / central clients]
    Mobile[Mobile app]
    TenantUI[Tenant UI]
  end

  subgraph api [Node API - Express]
    MW[Global middleware]
    TM[Tenant middleware ALS]
    RCentral[Central routes /api]
```

```
    RTenant[Tenant routes /api/tenant]
    RUserApp[User app /api/user-app]
    ERR[Error handler]
  end

  subgraph data [Data]
    MCentral[(MongoDB central DB)]
    MTenant[(MongoDB tenant_N DBs)]
  end

  subgraph aws [AWS]
    Cognito[Cognito]
    S3[S3]
    SQS[SQS]
    SNS[SNS push]
    SES[SES email]
  end

  Web --> api
  Admin --> api
  Mobile --> api
  TenantUI --> api
  api --> MCentral
  api --> MTenant
  Mobile --> Cognito
  api --> Cognito
  api --> S3
  api --> SQS
  Worker[SQS worker] --> SQS
  Worker --> SNS
  Worker --> SES
```

---

4. Application bootstrap

Entry point: `src/app.ts`.

1. Load `express-async-errors` so async route errors reach Express error middleware.
2. Apply global middleware: `helmet`, `cors` (credentials allowed), `morgan`, `cookie-parser`, JSON/urlencoded parsers, then **`tenantMiddleware`** (see §6).
3. Expose static files from `frontend/dist` and a **SPA fallback** `GET *` that serves `index.html` for non-`/api` paths (API handlers are registered afterward and still receive `/api` traffic via `next()`).
4. Mount route modules under `/api` (central), `/api/user-app`, and `/api/tenant`.
5. Global error handler returns `500` with `err.message`.
6. **`bootstrap()`**: `connectCentral()`, `registerCentralSchemas(conn)`, then `app.listen(config.port)`.

Configuration is centralized in `src/config/index.ts` (`dotenv`), including MongoDB names, JWT, AWS region/credentials, Cognito, S3, SQS, SES, optional `REDIS_URL`, and `CENTRAL_DOMAINS` (used for domain-oriented concepts elsewhere; routing is not subdomain-based on Node).

---

5. API surface and routing layout

- **Prefix**: All REST endpoints intended for this service live under **`/api`** (not hostname-per-tenant like the Laravel app).

- **Central**: Routers in `src/routes/*.routes.ts` — e.g. `auth`, `onboarding`, `organisations`, `masters`, `employees`, `plans`, `roles`, `settings`, `health`, etc.
- **User app**: `src/routes/user-app/index.ts` mounted at **`/api/user-app`** (bookings, travellers, feedback, vehicle tracking, trip purposes, users).
- **Tenant**: `src/routes/tenant/index.ts` mounted at **`/api/tenant`** — tenant login, then JWT-protected CRUD and live location endpoints.

Discovery: `GET /api` returns a short version payload; **`GET /api/health`** and **`GET /api/ready`** support probes; **`GET /api/docs`** / **`GET /api/docs.json`** serve Swagger.

---

6. Multi-tenancy model

6.1 Databases

- **Central**: MongoDB database name from `MONGODB_CENTRAL_DB` (default `vanloka_central`). Connection helper: `connectCentral()` / `getCentralConnection()` in `src/config/database.ts`.
- **Tenant**: One database per tenant: **`tenant_<tenantId>`**, via `getTenantConnection(tenantId)`. Connections are cached in a `Map` for reuse.

`createTenantDatabase` ensures initialization material exists when provisioning a tenant DB.

6.2 Request context (`x-tenant-id`)

`src/common/tenancy/tenant-middleware.ts` runs on every request:

- If header **`x-tenant-id`** is set, `AsyncLocalStorage` (`tenantStorage` in `tenant-context.ts`) stores `{ tenantId, isCentral: false }`.
- Otherwise `{ tenantId: null, isCentral: true }`.

Helpers `getTenantId()`, `isCentralRequest()`, and `getTenantContext()` read this context. Tenant routes additionally resolve tenant from header, query, body, JWT payload, or path segments depending on the handler (see `getTenantContext` in `tenant/index.ts`).

Laravel comparison (also documented in `API_ROUTES.md`): Laravel uses tenant **subdomains**; Node uses a **single host** and explicit **`x-tenant-id`** (or tenant id embedded in tenant JWT).

6.3 Schemas and models

- **Central models**: Mongoose schemas under `src/schemas/central/`, registered once on the central connection via `registerCentralSchemas(conn)` (`schemas/central/index.ts`).
- **Tenant models**: `src/schemas/tenant/`, registered on the tenant connection with `registerTenantSchemas(conn)` when a handler opens that connection.

This avoids registering the same model name on the wrong connection and keeps tenant data isolated by database.

---

7. Authentication and authorization

Implemented in `src/middleware/auth.middleware.ts` and route-specific usage.

Flow	Mechanism	Typical `Authorization`
Central web	HS256 JWT signed with `JWT_SECRET`, payload includes `type: 'web'`	`Bearer <jwt>`

Flow	Mechanism	Typical `Authorization`
Mobile	AWS **Cognito** access token verified by SDK; API then issues app JWT with `type: 'mobile'`	Cognito token on login route; subsequent calls often use issued JWT
Tenant portal	`POST /api/tenant/tenant-login` validates tenant DB user password; returns JWT with `type: 'tenant'` and `tenantId`	`Bearer <tenant jwt>`

`jwtAuth`: Tries local JWT verification first; on failure, attempts **Cognito** verification (`services/cognito.service.ts`) for mobile tokens. **`optionalAuth`** attaches user context if a valid token is present but does not require it.

Authorization (RBAC) is partially reflected in models (e.g. tenant roles/permissions); enforcement is per-route and not a single global guard layer.

---

8. Services layer (`src/services`)

Shared integrations and domain helpers:

- **`cognito.service`**: Token verification for mobile.
- **`s3.service`**: Object storage.
- **`sqs.service`**: Queue publish/consume patterns as used by the app.
- **`sns-push.service`**, **`ses.service`**: Push and email from worker or API paths.
- **`vehicle-location.service`**: Live vehicle location used by tenant tracking endpoints.

The worker (`src/workers/notification-worker.ts`) polls **SQS** (`SQS_NOTIFICATIONS_QUEUE_URL` / `SQS_QUEUE_URL`) and dispatches **SES** / **SNS** by message `type`.

---

9. Error handling and consistency

- Async errors: `express-async-errors`.
- Global Express error middleware returns JSON `{ error: message }` with status **500** for unhandled errors.
- Individual routes return **4xx** with ad-hoc body shapes (`error`, `status`, `message`, `success`, etc.) — clients should treat status codes as canonical; body shape may vary by route family.

---

10. Build and deployment

- **Local dev**: `npm run dev` runs `tsx watch src/app.ts`.
- **Production build**: `Dockerfile` multi-stage: `npm ci` → `tsc` → slim image runs `node dist/app.js` (does not bundle the SPA build unless your CI copies `frontend/dist`; the `build` script builds frontend + TS).
- **Port**: `PORT` (default **3000**).

---

11. Operational checklist (environment)

Minimum variables to review for a real deployment: `MONGODB_URI`, `MONGODB_CENTRAL_DB`, `JWT_SECRET`, `AWS_REGION` and credentials where used, `COGNITO_*`, `S3_BUCKET`, `SQS_*`, `SES_*`. Optional: `REDIS_URL`, `CENTRAL_DOMAINS`.

---

12. Key source map

Path	Responsibility
`src/app.ts`	Express app assembly, static SPA, route mounts
`src/config/`	Env config, Mongo connections, Swagger spec builder
`src/routes/`	HTTP routers (central, user-app, tenant)
`src/middleware/`	`jwtAuth`, `optionalAuth`
`src/common/tenancy/`	ALS tenant context, middleware
`src/schemas/central`, `src/schemas/tenant`	Mongoose models registration
`src/services/`	AWS and domain services
`src/workers/`	Long-running SQS consumer

13. Related documents

- `API_ROUTES.md` — Laravel vs Node path comparison and implementation coverage.
- `IMPROVEMENTS.md` — Further improvements backlog (API consistency, security, testing, observability, worker hardening).
- **Swagger** — Authoritative interactive listing for many (not necessarily all) exposed operations: `/api/docs`.